

Topic	Subtopic	Keywords
Pointer		C Pointers - javatpoint
Function		
Basic Data Structures	Array	
	Linked List	Singly Linked List Circular Linked List Doubly Linked List
	Stack	LIFO, Push, Pop
	Queue	FIFO, Enqueue, Dequeue
	Hash Table	Hash Function, Key, Hash values
	Tree	BST : Binary Tree Set 1 (Introduction) - GeeksforGeeks Node → Value, Left Pointer, Right Pointer
	Heap	Special BST Min Heap and Max Heap Min Heap: Root contains minimum value and always parent has value less or equal to both of its children Max Heap: Root contains max value, parent is greater or equal to both children
	Graph	Vertices, Edges Directed and Undirected graphs
Basic data types of C	Int, float, double, char	
	Unsigned int	- Can hold zero and positive numbers - In n-bit integers, range = 0 to $2^n - 1$
	Signed int	- Can hold negative, zero and positive numbers
	Long int	- Occupies 4 bytes (32 bits) of memory to store an integer value.
Struct		User defined data type that can group different types into a single type
Call by Value and Call by Reference in C - javatpoint	Call by value	- Copy of the value is passed - Changes made inside the function are limited to the function only - Different memory location
	Call by reference	- Address of the value is passed - Changes made inside the function gets reflected outside - Same memory location
Conditional Statements	If, If-else, Switch	

		- Data structure that the user defines
	Object	There's no memory allocation until we create object of a class
	Method	Functions(defined inside a class) that we use to describe the behavior of the objects of that class
	Inheritance	- One class inherits/acquires the attributes and methods of another class. - Parent class and Child class
	Encapsulation	- A way to ensure security - Binds or wraps like a capsule - Done using access modifiers (e.g. public, private, package private etc)
	Polymorphism	- Method overloading: Same name, different signature. - Method overriding: Same name, same signature. Used in inheritance.
	Abstraction	- Hiding internal details and showing functionality - Abstract class cannot be instantiated
	Static	- Mainly used for memory management - The static variable gets memory only once in the class area at the time of class loading
	Final	- Final variable: Can not change the value - Final method: Can not override it - Final class: Can not extend it
Database Relations	One to one	- Both tables can have only one record on each side of the relationship. - Each primary key value relates to none or only one record in the related table.
	One to many	- The primary key table contains only one record that relates to none, one, or many records in the related table.
	Many to many	- Each record in both tables can relate to none or any number of records in the other table - Requires a third table , called an associate or linking table, because relational systems cannot directly accommodate the relationship.
SQL and No SQL	SQL	- Structured Query Language for RDMS - Computer language for storing, manipulating and retrieving data stored in a relational database. - RDMS like MySQL, MS Access, Oracle, Postgres and SQL Server use SQL as their standard database language - Vertical scaling
	No SQL	- Non relational - Not in tabular format - Horizontal Scaling - Unstructured JSON
ACID properties	A = Atomicity	The entire transaction takes place at once or doesn't happen at all
	C = Consistency	The db must be consistent before and after the transaction

	D = Durability	The changes of a successful transaction occurs even if the system failure occurs
Bitwise Operator in C	AND, OR, XOR, NOT, Left Shift, Right Shift	Bitwise Operator in C - javatpoint
2's complement in C		2's complement of a = !a + 1
Dynamic memory allocation	Static vs Dynamic	Static = Array, memory is allocated at compile time Dynamic = Linked List, memory is allocated at run time
	Malloc	Dynamic Memory Allocation in C - javatpoint - Allocated single block of requested memory
	Free	- Frees the dynamically allocated memory
Sorting	Merge Sort	Merge Sort - GeeksforGeeks Merge sort in 3 minutes - Divide and Conquer - Divide the given array in half, keep dividing resulting arrays in halves until size 2 - Merging: Keeping two pointers and compare the elements of the subarrays from the start
	Quick Sort	QuickSort - GeeksforGeeks Quick sort in 4 minutes - Divide and Conquer - Pivot All items on the left of the pivot are smaller than the pivot All items on the right of the pivot are larger than the pivot - Recursive on the left sub array and right sub array
	Insertion Sort	Insertion sort in 2 minutes - Starting from left to right - Compare the item with the items in its left until it's in the correct position
	Selection Sort	Selection sort in 3 minutes - We have a sorted partition, we will keep adding current minimum in our sorted partition - Selecting the minimum from the unsorted array and adding it to the sorted partition
	Bubble Sort	Bubble sort in 2 minutes - Take every consecutive items and sort them - After every iteration the highest number will bubble to the end of the array

	<table><thead><tr><th></th><th>Best</th><th>Average</th><th>Worst</th></tr></thead><tbody><tr><td>Selection Sort</td><td>$\Omega(n^2)$</td><td>$\theta(n^2)$</td><td>$O(n^2)$</td></tr><tr><td>Bubble Sort</td><td>$\Omega(n)$</td><td>$\theta(n^2)$</td><td>$O(n^2)$</td></tr><tr><td>Insertion Sort</td><td>$\Omega(n)$</td><td>$\theta(n^2)$</td><td>$O(n^2)$</td></tr><tr><td>Heap Sort</td><td>$\Omega(n \log(n))$</td><td>$\theta(n \log(n))$</td><td>$O(n \log(n))$</td></tr><tr><td>Quick Sort</td><td>$\Omega(n \log(n))$</td><td>$\theta(n \log(n))$</td><td>$O(n^2)$</td></tr><tr><td>Merge Sort</td><td>$\Omega(n \log(n))$</td><td>$\theta(n \log(n))$</td><td>$O(n \log(n))$</td></tr><tr><td>Bucket Sort</td><td>$\Omega(n+k)$</td><td>$\theta(n+k)$</td><td>$O(n^2)$</td></tr><tr><td>Radix Sort</td><td>$\Omega(nk)$</td><td>$\theta(nk)$</td><td>$O(nk)$</td></tr><tr><td>Count Sort</td><td>$\Omega(n+k)$</td><td>$\theta(n+k)$</td><td>$O(n+k)$</td></tr></tbody></table>		Best	Average	Worst	Selection Sort	$\Omega(n^2)$	$\theta(n^2)$	$O(n^2)$	Bubble Sort	$\Omega(n)$	$\theta(n^2)$	$O(n^2)$	Insertion Sort	$\Omega(n)$	$\theta(n^2)$	$O(n^2)$	Heap Sort	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n \log(n))$	Quick Sort	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n^2)$	Merge Sort	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n \log(n))$	Bucket Sort	$\Omega(n+k)$	$\theta(n+k)$	$O(n^2)$	Radix Sort	$\Omega(nk)$	$\theta(nk)$	$O(nk)$	Count Sort	$\Omega(n+k)$	$\theta(n+k)$	$O(n+k)$
	Best	Average	Worst																																						
Selection Sort	$\Omega(n^2)$	$\theta(n^2)$	$O(n^2)$																																						
Bubble Sort	$\Omega(n)$	$\theta(n^2)$	$O(n^2)$																																						
Insertion Sort	$\Omega(n)$	$\theta(n^2)$	$O(n^2)$																																						
Heap Sort	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n \log(n))$																																						
Quick Sort	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n^2)$																																						
Merge Sort	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n \log(n))$																																						
Bucket Sort	$\Omega(n+k)$	$\theta(n+k)$	$O(n^2)$																																						
Radix Sort	$\Omega(nk)$	$\theta(nk)$	$O(nk)$																																						
Count Sort	$\Omega(n+k)$	$\theta(n+k)$	$O(n+k)$																																						
Graph Algorithms	BFS																																								
	DFS																																								
	Prim MST																																								
	Kruskal MST																																								
	Dijkstra	Single Source Shortest path																																							
	Bellman Ford																																								
	Floyd Warshall																																								
	Topological Sort	Kahn's Algorithm																																							
IP address MAC address																																									
OSI Model	Layers	1. Application →Data →HTTP, HTTPS, DNS 2. Presentation →Data 3. Session →Data 4. Transport → Segment →TCP, UDP 5. Network →Packet →IP, ICMP 6. Data Link → Frame →Ethernet 7. Physical → Bit → Cables																																							
TCP vs UDP	TCP	- Transmission Control Protocol - 3 way handshake → SYN, SYN/ACK, ACK - Connection oriented - Comparatively slower																																							

		- Connectionless - Comparatively faster, real-time services - Retransmission not possible, unreliable - DNS
Flow Control vs Congestion Control	Flow control	- Traffics that are flows from sender to receiver - Handled by data link layer and transport layer
	Congestion control	- Controlled entering the new network - Handled by network layer and transport layer
SOLID		Single Responsibility Principle Open Close Principle Liskov Substitution Principle Interface Segregation Principle Dependency Inversion Principle
Design Patterns	Creational Patterns	Factory Abstract Factory / Factory of Factory Builder → Director Class Singleton → Single class that makes sure that only one object of the class gets created, private constructor
	Structural Patterns	Adapter Decorator Composite
	Behavioral Patterns	State Strategy Observer / Event Subscriber-Listener Chain of Responsibility Mediator